

CppHDL Best Practices

CppHDL is a C++ RTL framework for writing synthesizable hardware models while keeping the code executable as normal C++. The main advantages are:

- You write RTL in a familiar general-purpose language with templates, types, functions, and normal build tools.
- The same source can run as a fast native C++ model and can also be converted to SystemVerilog.
- Testbenches can be ordinary C++ programs, so random tests, reference models, file I/O, and debugging are simple.
- Large parameterized designs can use C++ templates instead of large preprocessor-heavy SystemVerilog code.
- CppHDL encourages small modules, explicit registers, and clear combinational/sequential separation.

Mapping of SystemVerilog Expressions to C++

CppHDL code should be written as a direct C++ mapping of synthesizable SystemVerilog RTL. Continuous assignments and module port connections go into the `_assign()` section. This section runs only once, before the work cycle starts, and binds assignments that are used later during simulation and SystemVerilog generation. The `_ASSIGNxxx()` macros are only allowed in `_assign()`.

SystemVerilog:

```
1 assign out = a + b;
2 child.valid_i = valid;
3 child.data_i = data[i];
4 child.result_i[i] = result[i] ^ mask;
```

CppHDL:

```
1 _PORT(u<32>) out = _ASSIGN(a_in() + b_in());
2
3 void _assign()
4 {
5     child.valid_in = _ASSIGN(valid_reg);
6     for (int i = 0; i < LANES; i++) {
7         child.data_in[i] = _ASSIGN_I(data_reg[i]);
8         child.result_in[i] = _ASSIGN_I(result_reg[i] ^ mask_reg);
9     }
10 }
```

Use `_ASSIGN(expr)` for expressions. Use `_ASSIGN_REG(reg_or_signal)` for direct storage bindings such as registers, logic values, memories, or ports whose final object reference is enough. Use `_ASSIGN_COMB(comb_func())` when assigning the result of a CppHDL combinational function. Even though `_ASSIGN_COMB()` captures the returned object by reference, the `comb_func()` call itself is still executed on demand when the port value is read. For loop-indexed assignments use `_ASSIGN_I`, `_ASSIGN_REG_I`, `_ASSIGN_COMB_I`, or the indexed forms such as `_ASSIGN_INDEXED((i,j,k), expr)` and `_ASSIGN_REG_INDEXED((i,j,k), object[i][j][k])`.

All SystemVerilog `always_ff` blocks for one module map into one CppHDL `_work(bool reset)` method. `_work()` computes next register values. It may contain the logic that would be split across several `always_ff` blocks in SystemVerilog.

SystemVerilog:

```

1  always_ff @(posedge clk) begin
2      if (reset) count <= '0;
3      else if (enable) count <= count + 1;
4  end
5
6  always_ff @(posedge clk) begin
7      if (reset) valid <= 1'b0;
8      else valid <= enable;
9  end

```

CppHDL:

```

1  reg<u<8>> count_reg;
2  reg<u1> valid_reg;
3
4  void _work(bool reset)
5  {
6      if (reset) {
7          count_reg.clr();
8          valid_reg.clr();
9          return;
10     }
11
12     if (enable_in()) {
13         count_reg._next = count_reg + u<8>(1);
14     }
15     valid_reg._next = enable_in();
16 }

```

SystemVerilog `always_comb` blocks map to CppHDL combinational functions, usually named `*_comb_func()`. They calculate temporary combinational values from current inputs and current register values. The usual style is to store the result in a member variable and return it by reference, as shown in the root examples.

SystemVerilog:

```

1 always_comb begin
2     hit = valid && tag == req_tag;
3     read_data = hit ? line[word] : '0;
4 end

```

CppHDL:

```

1 bool hit_comb;
2 bool& hit_comb_func()
3 {
4     hit_comb = valid_reg && tag_reg == req_tag_in();
5     return hit_comb;
6 }
7
8 u<32> read_data_comb;
9 u<32>& read_data_comb_func()
10 {
11     read_data_comb = hit_comb_func() ? line_reg[word_in()] : u<32>(0);
12     return read_data_comb;
13 }
14
15 _PORT(u<32>) read_data_out = _ASSIGN_COMB(read_data_comb_func());

```

CppHDL commits registers and memories in the mandatory `_strobe()` method. `_strobe()` is executed recursively for each module at the end of each clock evaluation. Register `.strobe()` calls and memory `.apply()` calls are only allowed in `_strobe()`, not in `_assign()`, `_work()`, or comb functions.

```

1 void _strobe()
2 {
3     count_reg.strobe();
4     valid_reg.strobe();
5     memory.apply();
6 }

```

Think Like RTL

CppHDL is C++, but the model should still be written as RTL. Keep the same mental model as SystemVerilog:

- Ports describe module inputs and outputs.
- `reg<T>` is state.
- `_work(reset)` computes next state.
- `_strobe()` commits next state to current state.

- Combinational functions compute outputs from current inputs/state.

SystemVerilog:

```

1 always_ff @(posedge clk) begin
2     if (reset) begin
3         count <= '0;
4     end else if (enable) begin
5         count <= count + 1;
6     end
7 end
8
9 assign done = (count == LIMIT);

```

CppHDL:

```

1 reg<u<8>> count;
2 _PORT(bool) enable_in;
3 _PORT(bool) done_out = _ASSIGN(count == LIMIT);
4
5 void _work(bool reset)
6 {
7     if (reset) {
8         count.clr();
9         return;
10    }
11    if (enable_in()) {
12        count._next = count + u<8>(1);
13    }
14 }
15
16 void _strobe()
17 {
18     count.strobe();
19 }

```

Ports

Use `_PORT(type)` for module and interface ports. A port is called like a function to read its value:

```

1 _PORT(bool) valid_in;
2 _PORT(logic<32>) data_in;
3
4 if (valid_in()) {
5     word._next = data_in();
6 }

```

Output ports are usually initialized with `_ASSIGN_REG(...)`, `_ASSIGN_COMB(...)`, or `_ASSIGN(...)`:

```
1 reg<u1> ready_reg;
2 _PORT(bool) ready_out = _ASSIGN_REG(ready_reg);
3 _PORT(bool) empty_out = _ASSIGN(count == 0);
```

`_ASSIGN_REG`, `_ASSIGN`, and Indexed Expressions

`_ASSIGN_REG(x)` connects a port to a persistent variable. It is the best choice for registers and member variables.

```
1 reg<logic<32>> data_reg;
2 _PORT(logic<32>) data_out = _ASSIGN_REG(data_reg);
```

`_ASSIGN(expr)` connects a port to an expression. Use it for simple computed outputs:

```
1 _PORT(bool) fire_out = _ASSIGN(valid_in() && ready_in());
```

`_ASSIGN_I(expr)` and `_ASSIGN_REG_I(expr)` are used in loops where the loop index must be captured for each generated connection:

```
1 for (i = 0; i < N; ++i) {
2     out[i].valid_in = _ASSIGN_I(sel == i ? input.valid_in() : 0);
3 }
```

There are also `_J` and `_IJ` forms for nested loops.

Combinational Logic

For simple outputs, `_ASSIGN(...)` is enough. For larger combinational logic, write a member function that stores the result in a member variable and returns it by reference.

```
1 logic<32> result_comb;
2
3 logic<32>& result_comb_func()
4 {
5     result_comb = a_in() + b_in();
6     if (sub_in()) {
```

```

7     result_comb = a_in() - b_in();
8 }
9 return result_comb;
10 }
11
12 _PORT(logic<32>) result_out = _ASSIGN_COMB(result_comb_func());

```

Keep combinational functions side-effect-free except for assigning their own cached result variable.

Sequential Logic

Write next-state logic in `_work(reset)` and commit it in `_strobe()`.

```

1 void _work(bool reset)
2 {
3     if (reset) {
4         valid_reg.clr();
5         return;
6     }
7
8     valid_reg._next = next_valid;
9     data_reg._next = next_data;
10 }
11
12 void _strobe()
13 {
14     valid_reg.strobe();
15     data_reg.strobe();
16 }

```

Call nested module hooks explicitly:

```

1 void _work(bool reset)
2 {
3     child._work(reset);
4     state._next = child.data_out();
5 }
6
7 void _strobe()
8 {
9     child._strobe();
10    state.strobe();
11 }

```

CppHDL and SystemVerilog Look Similar

A valid-ready transfer in SystemVerilog:

```
1 assign out_valid = valid_reg;
2 assign out_data  = data_reg;
3
4 always_ff @(posedge clk) begin
5     if (reset) begin
6         valid_reg <= 1'b0;
7     end else if (!valid_reg out_ready) begin
8         valid_reg <= have_data;
9         data_reg  <= next_data;
10    end
11 end
```

The same idea in CppHDL:

```
1 _PORT(bool) valid_out = _ASSIGN_REG(valid_reg);
2 _PORT(logic<32>) data_out = _ASSIGN_REG(data_reg);
3 _PORT(bool) ready_in;
4
5 void _work(bool reset)
6 {
7     if (reset) {
8         valid_reg.clr();
9         return;
10    }
11    if (!valid_reg ready_in()) {
12        valid_reg._next = have_data;
13        data_reg._next = next_data;
14    }
15 }
```

Use C++ Templates for Parameters

Prefer templates for structural parameters such as data width, address width, number of ports, or FIFO depth.

```
1 template<size_t DATA_WIDTH, size_t DEPTH>
2 class Fifo : public Module
3 {
4     _PORT(logic<DATA_WIDTH>) write_data_in;
5     reg<u<clog2(DEPTH)>> write_ptr;
6 };
```

This keeps parameterized RTL type-safe and avoids many preprocessor tricks.

Interfaces

Use `Interface` for bundles containing signals of different directions. You do not need separate driver and responder interface types.

```
1 template<size_t DATAWIDTH>
2 struct ValidReadyIf : public Interface
3 {
4     _PORT(bool) valid_in;
5     _PORT(bool) ready_out;
6     _PORT(logic<DATAWIDTH>) data_in;
7 };
```

A source and sink can both use `ValidReadyIf`. A parent module or test wrapper can connect them with `assignIf()`:

```
1 VRDriver<32> driver;
2 VRResponder<32> responder;
3
4 void _assign()
5 {
6     assignIf(driver, responder, driver.source_out, responder.sink_in);
7 }
```

`assignIf()` performs the bidirectional assignment order needed when one side drives `valid/data` and the other side drives `ready`.

Arrays, Logic, and Bit Ranges

Use `logic<N>` for arbitrary-width bit vectors and `u<N>` for unsigned integers up to 64 bits. Use `.bits(hi, lo)` and `operator[]` for bit slicing.

```
1 logic<64> word;
2
3 word.bits(31, 0) = low_word;
4 word.bits(63, 32) = high_word;
5 word[0] = parity_bit;
```

Use `array<T,N>` for packed arrays and `memory<T,WIDTH,DEPTH>` for memories.

Keep Synthesizable Code Simple

CppHDL can run any C++ in native simulation, but converted RTL should use a synthesizable subset:

- Variables can be declared inside methods/functions, but put all declarations at the very beginning of the method/function.
- Avoid dynamic allocation in RTL models.
- Avoid STL containers in synthesizable state.
- Prefer fixed-size CppHDL types: `u<>`, `logic<>`, `array<>`, `memory<>`, and structs.
- Put file I/O, randomization, and reference models in inline tests, not in RTL modules.

Inline Tests

Examples and tests usually keep the first test in the same `.cpp` file as the RTL model. This is the CppHDL inline test practice: the file contains synthesizable modules first, then a `// CppHDL INLINE TEST` section with ordinary C++ test code.

An inline test normally does four things:

- Instantiates the CppHDL model.
- Drives input ports with C++ values.
- Runs the RTL cycle hooks: `_assign()`, `_work(reset)`, and `_strobe()`.
- Checks outputs and internal behavior against expected values or a C++ reference model.

Minimal shape:

```
1  // CppHDL INLINE TEST //////////////////////////////////////
2
3  long sys_clock = -1;
4
5  void tick(MyModule& dut, bool reset)
6  {
7      dut._assign();
8      dut._work(reset);
9      ++sys_clock;
10     dut._strobe();
11 }
12
13 void TestMyModule()
14 {
15     MyModule dut;
16
17     tick(dut, true);
18     tick(dut, false);
```

```
19     TEST_ASSERT(dut.done_out() == false);
20 }
21
```

Keep randomization, file I/O, scoreboards, and reference models in the inline test, not in synthesizable RTL classes. This keeps the RTL clean while still allowing strong native C++ tests.

sys_clock

Every CppHDL inline test must define:

```
1 long sys_clock = -1;
```

`sys_clock` is the global simulation clock used by CppHDL to cache and refresh combinational values. It must be incremented once per simulated clock edge before or around `_strobe()`, as shown in the examples:

```
1 dut._assign();
2 dut._work(reset);
3 ++sys_clock;
4 dut._strobe();
```

Without `sys_clock`, inline tests either fail to link or do not update cached combinational expressions correctly. This is why all examples define it in the test section, even when the RTL model itself has no explicit clock port.

Build and Run Tests

The `examples/` and `tests/` folders are built by CMake. Each `.cpp` file becomes one executable target, and the post-build command runs the `cpphdl` tool to generate SystemVerilog. Configure once:

```
1 cmake -S . -B build
```

Build one test:

```
1 cmake --build build --target interface_ValidReady
```

Run the native CppHDL test:

```
1 ./build/tests/interface_ValidReady --noveril
```

Run the full test, including the Verilator part when the inline test supports it:

```
1 ./build/tests/interface_ValidReady
```

For a new test, place it under `tests/<group>/<Name>.cpp`. The target name is derived from the relative path, so `tests/interface/ValidReady.cpp` becomes `interface_ValidReady`.

Conclusion

CppHDL runs RTL directly as native C++. No translator is used for native simulation, so compile/debug/test cycles are fast and use ordinary C++ tooling. For large projects, native CppHDL simulation can run about 10 times faster than Verilator and about 100 times faster than traditional SystemVerilog simulators, while the same RTL source can still be converted to SystemVerilog when needed.